

ABC450 E - Fibonacci String 题解

题意概括

给定两个字符串 X, Y ，定义：

- $S_1 = X$
- $S_2 = Y$
- $S_i = S_{i-1} + S_{i-2} \ (i \geq 3)$

这里的 $+$ 表示字符串拼接，而且顺序是先 S_{i-1} ，再 S_{i-2} 。

接下来有 Q 次询问。每次给出 L, R, C ，要求统计在 $S_{10^{18}}$ 的第 L 到第 R 个字符中，字符 C 出现了多少次。

解题思路

直接构造字符串显然不可能，因为下标已经达到 10^{18} 。

这题的关键是两件事：

1. 只需要关心“前缀”，而不是完整串。
2. 递推串虽然很长，但层数其实不多。

一、先把真正需要处理的串截到足够长

设 $\text{len}[i]$ 表示 S_i 的长度，那么

- $\text{len}[1] = |X|$
- $\text{len}[2] = |Y|$
- $\text{len}[i] = \text{len}[i-1] + \text{len}[i-2]$

长度增长和斐波那契数列同阶，所以非常快。我们只要找到最小的 top ，满足：

- $\text{len}[\text{top}] \geq 10^{18}$

就够了。

原因是对于任意 $i \geq 2$ ， S_{i+1} 的前 $\text{len}[i]$ 个字符恰好就是整个 S_i ，因为

- $S_{i+1} = S_i + S_{i-1}$

所以当 $\text{len}[\text{top}] \geq 10^{18}$ 时， $S_{10^{18}}$ 的前 10^{18} 个字符，和 S_{top} 的前 10^{18} 个字符完全一样。后面的更长递推已经不会影响询问区间。

于是问题就变成：

- 处理 S_{top} 的前缀统计

二、把区间询问改成两个前缀询问

如果我们能求出：

- $\text{prefix}(\text{pos}, c)$ = 在 S_{top} 的前 pos 个字符里，字符 c 出现了多少次

那么每个询问的答案就是：

- $\text{prefix}(R, c) - \text{prefix}(L - 1, c)$

所以核心只剩下如何快速求 $\text{prefix}(\text{pos}, c)$ 。

三、递归拆前缀

因为

- $S_i = S_{i-1} + S_{i-2}$

所以查询 S_i 的前 pos 个字符时，只有两种情况：

1. 如果 $\text{pos} \leq \text{len}[i - 1]$ ，那么答案完全在左半段 S_{i-1} 里。
2. 如果 $\text{pos} > \text{len}[i - 1]$ ，那么答案等于：
 - 整个 S_{i-1} 中字符 c 的出现次数；
 - 再加上 S_{i-2} 的前 $\text{pos} - \text{len}[i - 1]$ 个字符中的出现次数。

于是我们需要再预处理：

- $\text{cnt}[i][c]$ ：整个 S_i 中字符 c 的出现次数

转移也很直接：

- $\text{cnt}[i][c] = \text{cnt}[i - 1][c] + \text{cnt}[i - 2][c]$

四、边界串直接用前缀和回答

当递归拆到 $S_1 = X$ 或 $S_2 = Y$ 时，就不能再往下拆了。

所以我们对原串 X, Y 分别做字符前缀和：

- $\text{prex}[p][c]$ ： X 的前 p 个字符里，字符 c 的出现次数
- $\text{prey}[p][c]$ ： Y 的前 p 个字符里，字符 c 的出现次数

这样一来：

- 如果拆到 S_1 , 直接返回 $\text{prex}[\text{pos}][c]$
- 如果拆到 S_2 , 直接返回 $\text{prey}[\text{pos}][c]$

由于 top 只有大约 80 到 90 层, 每次前缀查询最多往下拆这么多次, 所以总复杂度完全够用。

正确性说明

1. 用 S_{top} 代替 $S_{10^{18}}$ 是正确的

我们取最小的 top , 满足 $\text{len}[\text{top}] \geq 10^{18}$ 。

对任意 $i \geq 2$, 都有

$$S_{i+1} = S_i + S_{i-1}$$

因此 S_{i+1} 的前 $\text{len}[i]$ 个字符恰好就是整个 S_i 。

不断向后推广可知: 对于任意 $j > \text{top}$, S_j 的前 $\text{len}[\text{top}]$ 个字符都和 S_{top} 完全相同。

而所有询问位置都不超过 $10^{18} \leq \text{len}[\text{top}]$, 所以无论题目写的是 $S_{10^{18}}$ 还是更后面的某一项, 询问到的那一段前缀都与 S_{top} 完全一致。

因此把问题改成在 S_{top} 上回答, 不会改变答案。

2. 前缀拆分公式是正确的

固定某个 $i \geq 3$, 因为

$$S_i = S_{i-1} + S_{i-2}$$

所以 S_i 的前 pos 个字符只有两种情况:

如果 $\text{pos} \leq \text{len}[i-1]$, 那么这个前缀完全落在左半段 S_{i-1} 里, 因此

$$\text{prefix}_i(\text{pos}, c) = \text{prefix}_{i-1}(\text{pos}, c)$$

如果 $\text{pos} > \text{len}[i-1]$, 那么前 pos 个字符由两部分组成:

- 整个 S_{i-1}
- 再加上 S_{i-2} 的前 $\text{pos} - \text{len}[i-1]$ 个字符

因此

$$\text{prefix}_i(\text{pos}, c) = \text{cnt}[i-1][c] + \text{prefix}_{i-2}(\text{pos} - \text{len}[i-1], c)$$

这正是代码里的转移。

3. 边界返回也是正确的

当我们一路拆到 $S_1 = X$ 或 $S_2 = Y$ 时，前缀统计就已经退化成普通字符串前缀计数。

而 prex 、 prey 记录的正是：

- 原串前若干个字符中，每个字母分别出现了多少次

所以：

- 到达 S_1 时返回 $\text{prex}[\text{pos}][c]$
- 到达 S_2 时返回 $\text{prey}[\text{pos}][c]$

就恰好等于定义中的答案。

4. 整个算法得到的就是每次询问的正确答案

由第 1 点可知，我们只需要在 S_{top} 上回答问题。

由第 2 点和第 3 点可知，函数 $\text{prefix}(\text{pos}, c)$ 会严格按照字符串拼接结构，把一个大前缀拆成若干个完整子串计数与一个边界前缀计数之和，因此它返回的正是 “ S_{top} 前 pos 个字符中 c 的出现次数”。

最后再用

- $\text{prefix}(R, c) - \text{prefix}(L - 1, c)$

就得到区间 $[L, R]$ 中字符 c 的出现次数。

所以整套算法正确。

复杂度

设 top 是满足 $\text{len}[\text{top}] \geq 10^{\{18\}}$ 的最小下标。

- 预处理长度、整串字符计数： $O(26 \cdot \text{top})$
- 预处理 X, Y 的前缀和： $O(26(|X| + |Y|))$
- 每次询问需要做两次前缀查询，每次最多向下拆 top 层，所以单次询问复杂度是 $O(\text{top})$

总时间复杂度为：

- $O(26(|X| + |Y|) + 26 \cdot \text{top} + Q \cdot \text{top})$

由于 top 只有大约 80 到 90，所以这在数据范围内是充足的。

空间复杂度为：

- $O(26(|X| + |Y|) + 26 \cdot \text{top})$

参考实现

```
#include<bits/stdc++.h>
using namespace std;

const int maxl = 10000;
const int maxm = 100;
const long long LIM = 10000000000000000LL;
const long long INF = 40000000000000000LL;

int prex[maxl + 5][26];
int prey[maxl + 5][26];
long long len[maxm + 5];
long long cnt[maxm + 5][26];

string x, y;
int top;

// 计算 S_top 的前 pos 个字符中, 字符 c 出现了多少次
long long solve_prefix(long long pos, int c){
    if(pos <= 0){
        return 0;
    }

    int id = top;
    long long ans = 0;

    while(true){
        if(id == 1){
            return ans + prex[pos][c];
        }
        if(id == 2){
            return ans + prey[pos][c];
        }

        // 当前前缀完全落在左半段 S_{id - 1} 中
        if(pos <= len[id - 1]){
            id--;
        } else {
            // 先整段加入 S_{id - 1} 的贡献, 再去右半段继续拆
            ans += cnt[id - 1][c];
        }
    }
}
```

```

        pos -= len[id - 1];
        id -= 2;
    }
}

int main(){
    cin >> x >> y;

    int nx = x.size();
    int ny = y.size();

    // 预处理 x 的字符前缀和
    for(int i=1; i<=nx; i++){
        for(int j=0; j<26; j++){
            prex[i][j] = prex[i - 1][j];
        }
        prex[i][x[i - 1] - 'a']++;
    }

    // 预处理 y 的字符前缀和
    for(int i=1; i<=ny; i++){
        for(int j=0; j<26; j++){
            prey[i][j] = prey[i - 1][j];
        }
        prey[i][y[i - 1] - 'a']++;
    }

    // 初始化 S_1、S_2 的长度和整串字符计数
    len[1] = nx;
    len[2] = ny;
    for(int j=0; j<26; j++){
        cnt[1][j] = prex[nx][j];
        cnt[2][j] = prey[ny][j];
    }

    // 只递推到长度第一次达到 10^18 为止
    top = 2;
    while(len[top] < LIM){
        top++;
        len[top] = min(INF, len[top - 1] + len[top - 2]);
    }
}

```

```
for(int j=0; j<26; j++){
    cnt[top][j] = min(INF, cnt[top - 1][j] + cnt[top - 2][j]);
}
}

int q;
cin >> q;

while(q--){
    long long l, r;
    char c;
    cin >> l >> r >> c;

    int ch = c - 'a';

    // 区间答案 = 右前缀 - 左边界前一个位置的前缀
    long long ans = solve_prefix(r, ch) - solve_prefix(l - 1, ch);
    cout << ans << '\n';
}

return 0;
}
```